Assignment 2:

Analytical Model Choice and Customer Segmentation

Group 5:

Gerald Abbey

Olamide Ibukunoluwa Yusuf

Nihal Vellaramkallingal Sulaiman

Souren Chowdhury

Course: DAMO-640-10: Machine Learning

Program: Master of Data Analytics, University of Niagara Falls Canada

Instructor: Touraj Banirostam

Date: May 24, 2026

Part A - Pre-Analysis and Hypotheses

Model Family Expectation

Before running any experiments, we hypothesized that an ensemble model would perform better for this problem. In particular, we theorized that a Random Forest classifier would outperform a Support Vector Machine (SVM) on this task. The *Retail_Customer_Insights_v2* dataset contains a mix of continuous numbers like average transaction value and categorical text such as region and preferred channel. Tree-based ensembles handle this kind of mixed data naturally. Because they split features using orthogonal decision boundaries rather than relying on geometric distance metrics.

On the other hand, SVMs map data points into a continuous high-dimensional space. When one-hot encoded categorical variables are introduced, you often end up with a sparse, noisy feature space. This tends to degrade the performance of the Radial Basis Function (RBF) kernel unless you spend an exhaustive amount of time tuning it.

Appropriate Evaluation Metrics

When it comes to predicting customer churn, looking only at accuracy is a major trap. Churn datasets are almost always heavily imbalanced; the majority class consists of retained customers, while only a small minority actually leaves. For instance, if 90% of the customers stay, a lazy model that just guesses "No Churn" every time will look 90% accurate, even though it completely fails to catch a single churner.

Because of this, we are using the F1-score the harmonic mean of precision and recall as our primary metric. It gives us a balanced look at how well the model catches actual churners without throwing out too many false alarms. We will also track ROC-AUC to see how stable the model’s discrimination thresholds are.

Expected Segmentation Structure

For the unsupervised learning portion of the project, we expect to find distinct customer behaviors driven by basic recency, frequency, and monetary patterns. Ideally, the clustering algorithms will slice the customer base into clear business categories. For example, we expect to see a "High-Value Loyal" group that spends heavily and interacts frequently. Alongside this, we expect an "At-Risk/Disengaged" group characterized by dropping visit frequencies and low transaction values.

Part B - Experimental Design

Data Splitting and Preparation

To ensure a fair and unbiased evaluation, we split the data into two parts - 70% training and 30% testing. We used a stratified split based on the target variable (churn_risk) to ensure that the minority churn class was distributed evenly across both the sets. We also locked in a random seed of 42 across all algorithms so that our results are completely reproducible.

Preprocessing Strategy

We built a clean preprocessing pipeline to handle data quirks without letting any test data leak into the training process. First, we dropped the customer_id column since unique identifiers don't offer any real predictive value. For missing values in the continuous features, we used median imputation because it handles extreme outliers much better than a standard average. Categorical features like region and preferred channel were converted using one-hot encoding. Most importantly, we organized all feature scaling (Standardization) and encoding steps into a Scikit-Learn Column Transformer and Pipeline. During running 5-fold cross-validation on the training data, this setup ensures that the standard scaler is fit only on the $k-1$ training folds and

simply applied to the remaining validation fold. This keeps validation data from accidentally leaking back and influencing the training math.

Part C - Classification Analysis

Model Training and Hyperparameter Tuning

As per the assignment guidelines, we evaluated two specific models: a Random Forest and an SVM with an RBF kernel. We limited hyperparameter tuning to just one setting per model. Specifically, we tested three values per model through a 5-fold cross-validation process optimized for the F1-score.

- **Random Forest:** Tuned the number of trees (n_estimators) using [50, 100, 200].
- **SVM:** Tuned the regularization parameter (C) using [0.1, 1.0, 10.0].

The cross-validation phases highlighted some clear stability differences. The Random Forest model was remarkably stable, showing very low variance (standard deviation) across all five validation folds. The SVM was far more sensitive to how the data subsets shifted, making it highly dependent on finding the perfect C penalty to balance its margin width against misclassification errors.

Performance and Stability Analysis

When we ran both models on the unseen 30% test set, we observed that the Random Forest clearly beat the SVM across every major metric, including F1-Score, Recall, and ROC-AUC.

Table 1

Model Performance Summary

Model	Precision	Recall	F1-Score	ROC-AUC
Random Forest	1	0.761	0.8642	0.985
SVM (RBF)	0.7838	0.63	0.6988	0.9316

This gap comes down to how the models are built. The Random Forest uses bagging (bootstrap aggregating), training a diverse group of decision trees on random subsets of data and features. Averaging these trees smooths out the model's overall variance. This stops it from overfitting to the day-to-day noise in retail data.

Meanwhile, the SVM struggled with the one-hot encoded categorical variables in our dataset. The RBF kernel calculates Euclidean distances between points in a transformed space. Breaking categorical traits out into multiple binary columns inflates the dimensionality of the feature space. This triggers the curse of dimensionality, which makes those distance calculations a lot less meaningful.

Business Perspective and Error Costs

In a real-world churn management setup, different mistakes carry different price tags. A **False Positive** happens when the model flags a loyal customer as a churn risk. The cost here is pretty minor—usually just the margins lost by giving them an unnecessary discount or retention offer.

A **False Negative**, however, means missing a customer who is actually about to walk out the door. The cost here is heavy: you permanently lose that customer’s lifetime value and have to spend a premium on marketing to acquire someone new to replace them. Because missing a churner is so much more expensive, maximizing Recall (catching as many true churners as possible) is our main business goal, even if it means taking a small hit on Precision.

Deployment Recommendation

Based on the metrics and the financial impact, we recommend deploying the Random Forest model. It offers stronger overall predictive power (better ROC-AUC) and steadier production stability (lower cross-validation variance). Just as importantly, it natively outputs

probability scores. In production, the business can easily lower the decision threshold below 0.5 to cast a wider net, catch more at-risk customers, and directly offset the high costs of False Negatives.

Part D - Unsupervised Learning and Segmentation

Methodology and Assumptions

To look for natural customer groupings without using pre-existing labels, we ran K-Means and DBSCAN clustering on our scaled dataset. These two algorithms approach data very differently:

- **K-Means** looks for round, convex, evenly dense clusters. It's a partition-based approach, meaning it forces every single data point into a cluster somewhere.
- **DBSCAN** searches for continuous, dense pockets of data separated by empty space. It doesn't force points into clusters. If a customer's behavior sits outside the dense neighborhoods defined by the epsilon (eps) parameter, the algorithm simply flags them as noise.

Clustering Evaluation

We tested K-Means with k values of 2, 3, 4, and 5, using Silhouette Scores. The aim is to check how tightly packed the clusters were and how well they were separated. We tested DBSCAN across several epsilon distances. The biggest takeaway from DBSCAN was that it threw a massive chunk of the dataset out as unclustered noise. This tells us something crucial about our data topology: retail customers don't live in isolated "islands" of behavior. Instead, purchasing habits, recency, and spending levels exist on a smooth, continuous spectrum.

Algorithm Suitability and Business Interpretation

For practical business use, K-Means is the clear winner here. Modern Customer Relationship Management (CRM) tools require total coverage; marketing teams need to put every customer into a specific track or campaign. DBSCAN's strict density requirements left too much of our customer base floating in an unusable "noise" category where they couldn't be targeted.

By using K-Means, we successfully sliced the continuous spectrum into clear and operational segments. Looking at the centroids of these clusters reveals two clear customer archetypes:

- **High-Value Loyals:** These customers show above-average spending with high visit frequency.
- **Disengaged:** These customers have low total spending and long gaps since their last purchase.

Even if these hard boundaries are mathematically forced rather than completely organic, they give marketing operations the structured framework they need to run targeted campaigns.

AI Tool Use Disclosure

During this project, we used Copilot as an analytical sounding board and formatting assistant. All core decisions regarding experimental setup, hyper parameter selections, and our initial hypotheses were developed entirely by the group members. Specifically, we used the AI to review our Python Pipeline structure to double-check that our cross-validation setup successfully prevented data leakage. Finally, after writing out our results and business analysis, we used the AI to help format and organize the text into a clean, APA-structured document that fits the grading rubric's clarity guidelines. We did not use any AutoML tools to build the models or run the code.